

PRÁCTICA CALIFICADA 2

1. En un nuevo proyecto en IntelliJ IDEA ApellidoNombre_PC2 crear un framework de automatización de pruebas que incluya las capas Hooks, Pages, Steps, Runner y Features, para el aplicativo web utilizado en laboratorio, el contenedor de software emp_auto.tar (8 puntos)
 - 1.1 Crear un caso de éxito y otro de no éxito que use todas las capas, estos casos deben ser distintos a los vistos en el laboratorio. (2 puntos)
 - 1.2 Ejecutar sus casos de prueba y enviar sus evidencias. (2 puntos)
 - 1.3 Utilizar un agente IA desde el IntelliJ IDEA para agregar una capa adicional de Datos a su framework de automatización de pruebas, puede usar el agente IA visto en clase Github Copilot o algún otro de su interés. (2 puntos)
 - 1.4 Ejecutar sus casos de prueba nuevamente con la capa adicional y enviar sus evidencias. (2 puntos)

2. La empresa SOFT GO desarrolla software para uno de sus clientes usando el marco de trabajo ágil SCRUM. Los desarrolladores Backend acaban de entregar al equipo de calidad de software el API REST para que se inicien las pruebas. El API REST ha sido implementado usando Java Spring Boot en base a los criterios técnicos de la Tabla 1.

Tabla 1

Historia de Usuario	Criterios técnicos API Rest
Como usuario, yo quiero ser capaz de mantener la información de los productos.	La operación POST /products crea un nuevo producto retornando el código 201. Body request: { <pre> "name": "string", "description": " string ", "stock": int, "price": double, "status": boolean }</pre> Response { <pre> "id": Long, "name": " string ", "description": " string ", "stock": int, "price": double, "status": boolean }</pre>

	La operación GET /products obtiene todos los productos retornando el código 200.
	La operación GET /products/id obtiene la información del producto id retornando el código 200.
	La operación PUT /products/id actualiza el producto retornando el código 200 . Body request: { <pre> "name": "string", "description": " string ", "stock": int, "price": double, "status": boolean }</pre> Response { <pre> "id": Long, "name": " string ", "description": " string ", "stock": int, "price": double, "status": boolean }</pre>
	La operación DELETE /products/id elimina el producto id. Retornando el código 200. En el response se obtiene “Producto eliminado correctamente.”
	La operación GET /products/id muestra en el response “Producto no encontrado.” y código 200 cuando el id no se encuentra.
	La operación PUT /products/id muestra en el response “Producto no encontrado.” y código 200 cuando el id no se encuentra.
	Para las operaciones POST y PUT cuando se ingresa un parámetro inválido en el body se retorna código de error 400 Bad Request.

Usted es el QA Engineer y se le ha encargado las siguientes actividades (12 puntos):

2.1 Preparación de ambiente de pruebas. Usar la imagen de contenedor de software proporcionada por el profesor e inicie el aplicativo. Enviar screenshots de toda la pantalla de los pasos para iniciar el aplicativo. (2 puntos)

- 2.2 Diseño de framework y casos de prueba automatizados. Usando un proyecto nuevo en Cypress Cypress_ApellidoNombre_PC2, implemente 3 casos de prueba para escenarios de éxito y 2 casos de prueba para escenarios de no éxito. (2 puntos)
- 2.3 Ejecución de casos de prueba. Ejecute de manera automatizada en modo consola y enviar los resultados obtenidos. (2 puntos)
- 2.4 Registro de defectos. Encuentre 2 defectos en base a los criterios técnicos de la tabla 1. Los defectos deben contar con su caso de prueba automatizado. (2 puntos)
- 2.5 Gestión de pruebas. Usando TestRail: crear un proyecto, casos de pruebas, plan de pruebas para los casos de pruebas automatizados del ítem 2.3 (2 puntos)
- 2.6 Gestión de pruebas. Usando TestRail: Continuando el ítem 2.5 crear la ejecución de pruebas y añadir sus resultados. Incluir los defectos. (2 puntos)

Enviar sus evidencias en formato imagen (screenshots de toda la pantalla no recortado) o videos. Puede usar Word o pdf para su envío.

Nota:

Para la creación de escenarios de no éxito de casos de pruebas automatizadas de API considerar el uso de **failOnStatusCode: false**. Evita que nuestra prueba falle automáticamente cuando la respuesta HTTP devuelve un error (códigos 4xx o 5xx).

Ejemplo:

```
it('API POST Request - Shouldnt create a client without a direction', () => {  
  const userPayload = {  
    name: "Ado Luna",  
    email: "ado.luna@esan.edu.pe"  
  };  
  cy.request({  
    method: 'POST',  
    url: 'http://localhost:8080/clients',  
    failOnStatusCode: false,  
    body: userPayload  
  }).then((response) => {  
    cy.log(response);  
    expect(response.status).to.eq(400);  
  });  
  cy.wait(2000);  
});
```